



brg

Never explain yourself twice.

A git-style CLI orchestrator for switching between AI coding tools – Claude Code, Codex, and whatever the community adds next – without losing the thread of what you were building.

You shouldn't have to re-explain your own project.

Every developer using more than one AI coding CLI runs into the same wall: switch from Claude Code to Codex – or back – and the new session knows nothing. No decisions, no open threads, no "why we did it this way." You re-type the same context, every single time.

What brg does about it

- Carries context with you. `brg switch claude` hands off to Claude Code with your project's rolling context already loaded – the way `git` carries history instead of making you retype it.
 - Snapshots progress on demand. `brg checkpoint "message"` records where the project stands, `git-commit` style – by hand, or auto-generated from the session you're leaving.
 - Starts clean when you want it to. `brg switch codex --fresh` skips context entirely for a throwaway experiment.
 - Never touches your credentials. `brg` shells out to each tool's own login flow – it doesn't store or see a single API key.
 - Stays fully local and plain-text. No database, no server, no lock-in – just Markdown and JSON files you can open, diff, and read yourself.
-

The bigger goal

`brg` is built for a world where the "best AI coding tool" keeps changing – and where developers, reasonably, use more than one. We want `brg` to be the neutral layer underneath that churn: the thing that stays yours no matter which tool is having a good month. That only works as an open, community-run standard, not a single vendor's walled garden – which is why this project is MIT-licensed and built from day one around a plug-in `adapter` pattern so anyone can add support for the next tool without touching the core.

What's in this handbook

01	Quickstart	install → setup → init → switch
02	Core concepts	context, checkpoints, sessions, handoff mode
03	Command reference	every command, every flag, real examples
04	Configuration	.brg/config.yaml settings
05	.brg/ directory	what's stored, in plain files
06	Troubleshooting	common errors and fixes
07	Contributing	how to add a tool adapter, join the project

This handbook covers brg 0.1.0 (published as brg-cli on npm). For the living, most up-to-date reference, see [docs/USER_GUIDE.md](#) in the repository – this PDF is a snapshot of it.

From zero to switched in four commands

```
# 1. install
$ npm install -g brg-cli

# 2. install/authenticate the AI CLIs you use
$ brg setup

# 3. initialize brg inside your project
$ brg init
✓ Initialized .brg/ in /path/to/your-project

# 4. hand off to Claude Code with context loaded
$ brg switch claude
```

brg setup

Idempotent wizard – detects what's already installed and logged in per tool, and only acts on what's missing. Safe to re-run anytime.

brg init

Creates .brg/ in the current directory: context.md, config.yaml, and an empty sessions/ folder. Also idempotent.

Requires Node.js 18+. The npm package is published as `brg-cli` (the bare name `brg` was already taken on the registry) but the command it installs is just `brg`.

Four ideas, borrowed from git

If you're comfortable with git's mental model, this will feel familiar – brg deliberately borrows it.

Context – the running summary

A human-readable rolling summary of your project – what's been done, key decisions, open threads – living in one plain Markdown file, `.brg/context.md`. This is what gets handed to the next tool on brg switch.

Checkpoints – snapshots in time

A checkpoint is a snapshot of where the project stands right now, the same way git commit snapshots your repo. brg checkpoint "message" appends a line to `context.md` and writes a structured record under `sessions/`. brg switch also writes one automatically before handing off, so nothing is lost even if you forget.

Sessions – the structured record

One JSON file per checkpoint under `.brg/sessions/`, named after its timestamp: `{ timestamp, tool, message, contextSnapshot }`. brg log reads every session file and prints them as a timeline.

Handoff mode – full control, then brg steps aside

brg spawns the target AI CLI as a child process with full terminal control, then exits – from that point on you're talking directly to claude or codex, no brg process in between. Before handing off, brg auto-checkpoints against whatever tool you were leaving, in a three-tier fallback: (1) ask that tool to summarize its own session, (2) read its on-disk transcript directly if that's unavailable, (3) fall back to a plain message. Nothing about this ever blocks the switch.

Every command, at a glance

COMMAND	WHAT IT DOES
<code>brg setup</code>	Interactive wizard to install/authenticate supported AI CLIs. Idempotent.
<code>brg tools list</code>	Lists registered AI CLIs and their install/auth state. Read-only.
<code>brg init</code>	Creates <code>.brg/</code> in the current project directory. Idempotent.
<code>brg switch <tool></code>	Hands off to an AI CLI, carrying project context with you.
<code>brg switch <tool> -f, --fresh</code>	Same, but skips context – starts a completely clean session.
<code>brg checkpoint <message></code>	Snapshots current state with a message, like <code>git commit</code> .
<code>brg log</code>	Prints a timeline of checkpoints, most recent first.
<code>brg status</code>	Shows active tool, last checkpoint, context size, today's count.
<code>brg context show</code>	Prints the current <code>.brg/context.md</code> to stdout.
<code>brg --version</code>	Prints the installed version.
<code>brg --help</code>	Shows all commands; <code>brg <command> --help</code> shows flags for one.

The following pages walk through each command in full detail, with flags and real terminal output. This mirrors `docs/USER_GUIDE.md` in the repository.

brg setup

Interactive wizard that installs and authenticates the AI CLIs you want to use. For each tool, in order: detect install → install if missing → detect login → trigger login if missing. Re-running only acts on what's still missing – brg never handles credentials itself.

```
$ brg setup
brg setup – pick which AI CLIs to set up.

  claude      Claude Code
  codex       Codex

Which tools do you want set up? (comma-separated, or "all") claude

Claude Code
  ✓ already installed
  ✓ already authenticated

✓ Setup complete. Run "brg init" inside a project to get started.
```

TOOL	INSTALL COMMAND	LOGIN COMMAND
claude	<code>npm i -g @anthropic-ai/claude-code</code>	<code>claude login</code>
codex	<code>npm i -g @openai/codex</code>	<code>codex login</code>

brg tools list

Lists every registered tool adapter and its current state. Read-only.

```
$ brg tools list
claude      Claude Code    installed, authenticated
codex       Codex          not installed
```

brg init

Creates `.brg/` in the current working directory: `context.md` (starter header), `config.yaml` (default `contextStrategy: ai-assisted`), and an empty `sessions/` directory. Idempotent – does nothing if `.brg/` already exists, rather than overwriting anything.

```
$ brg init
✓ Initialized .brg/ in /path/to/your-project

$ brg init # running it again
.br/ already exists – nothing to do.
```

brg switch <tool>

Unless `--fresh` is passed, auto-checkpoints against whichever tool your last session belongs to, reads `.brg/context.md`, then hands off full terminal control to `<tool>` – `brg` exits once the target tool starts. `<tool>` must already be installed (`brg setup` first).

FLAG	DEFAULT	MEANING
<code>-f, --fresh</code>	off	Skip auto-checkpoint and context entirely – start the target tool with a clean session.

```
$ brg switch nosuchtool
brg: unknown tool "nosuchtool". Known tools: claude, codex

$ brg switch codex
brg: Codex is not installed. Run "brg setup" first.
```

brg checkpoint <message>

Snapshots the current project state: appends a line to `context.md` and writes a new file under `sessions/`. Requires `.brg/` to exist.

FLAG	DEFAULT	MEANING
<code>--tool <name></code>	<code>defaultTool</code> or <code>"unknown"</code>	Which tool this checkpoint is attributed to.

```
$ brg checkpoint "wired up the auth middleware" --tool claude
✓ Checkpoint saved.

# appended to context.md, enriched with git branch + diffstat when available:
- [2026-08-10T11:53:41.179Z] claude: wired up the auth middleware
  (main, 3 files changed, 42 insertions(+), 5 deletions(-))
```

brg log

Prints every checkpoint as a one-line timeline, most recent first.

```
$ brg log
2026-08-10T11:53:42.315Z  codex  moved to codex for the refactor
2026-08-10T11:53:41.187Z  unknown first pass on the parser
```

brg status

A quick snapshot: configured defaultTool, time since last checkpoint, current context.md size, and today's checkpoint count. Safe to run anywhere – reports "not a brg project" instead of erroring.

```
$ brg status
active tool:      (not set)
last checkpoint: just now
context.md size: 220 bytes
checkpoints today: 2
```

brg context show

Prints the raw contents of .brg/context.md to stdout – useful for piping, or just reading without opening an editor.

```
$ brg context show
# Project Context

Rolling summary – what's been done, key decisions, open threads.

- [2026-08-10T11:53:41.179Z] claude: first pass on the parser
- [2026-08-10T11:53:42.315Z] codex: moved to codex for the refactor
```

.brg/config.yaml

Project-level settings, created by `brg init`. Plain YAML – edit it directly, no `brg config` command yet.

KEY	DEFAULT	MEANING
contextStrategy	ai-assisted	How a checkpoint's line gets generated. <code>ai-assisted</code> tries the active tool's own summary first, falls back to its raw transcript, then to your message as-is. <code>manual</code> skips straight to the last step – fully offline, zero dependencies.
defaultTool	unset	Which tool <code>brg checkpoint</code> attributes to when <code>--tool</code> isn't passed, and what <code>brg status</code> reports as active.

Plain files, on purpose

No database, no server – everything git-diffable and human-readable. Open `context.md` in a text editor and read your project history without ever running `brg`.

```
.brg/
├─ context.md      rolling summary – injected on `brg switch`
├─ config.yaml     contextStrategy, defaultTool
├─ sessions/
│   └─ <ISO-timestamp>.json
│       { timestamp, tool, message, contextSnapshot }
```

Common errors, quick fixes

- "no .brg/ directory found. Run brg init first." – you ran a command that needs .brg/ outside an initialized project. brg status is the one exception – it reports "not a brg project" instead of erroring.
- "unknown tool <name>. Known tools: claude, codex" – check spelling; it's the short adapter name (claude), not the display name.
- "<Tool> is not installed. Run brg setup first." – the target tool's binary isn't on PATH yet. Run brg setup or install it manually.
- brg setup says a tool isn't authenticated, but I know I logged in – detection checks fixed credential file paths; if that tool moved where it stores credentials, open an issue.
- brg checkpoint feels slow – that's the ai-assisted strategy shelling out to your AI CLI for a summary. Set contextStrategy: manual for instant, fully offline checkpoints.
- npm command not found after install – confirm npm's global bin directory is on PATH (npm config get prefix).
- Why is the package brg-cli but the command brg? – npm's registry blocks the bare name brg as too similar to an existing package. The bin field still maps to brg.

Full reset

```
$ npm uninstall -g brg-cli # remove brg itself
$ rm -rf .brg/             # remove one project's brg data
```

Built to outlive any one AI tool.

brg only supports Claude Code and Codex today – but the whole point of the adapter pattern is that it doesn't stay that way because of us. It grows because of you.

Add support for a new tool

Adding a CLI – Gemini CLI, OpenCode, whatever's next – means writing exactly one file under `src/tools/` that implements the `ToolAdapter` interface, and registering it in `registry.ts`. No changes needed anywhere else in the codebase – that's the whole design.

- Bug reports and feature ideas – open an issue
 - Docs fixes – always welcome, always small enough to review fast
 - New tool adapters – the highest-leverage contribution there is
-

Repository	Install	License
github.com/brghq/brg	<code>npm install -g brg-cli</code>	MIT – free, forever

See `CONTRIBUTING.md` and `CODE_OF_CONDUCT.md` in the repository for local setup, the PR flow, and how we work together. This handbook mirrors `docs/USER_GUIDE.md` – for the always-current reference, read it straight from source.